QuillAudits

AUDIT REPORT

October 2026

For

Lendary

Table of Content

Executive Summary 04

Number of Security Issues per Severity 06

Summary of Issues 07

Checked Vulnerabilities 09

Techniques and Methods 11

Types of Severity 13

Types of Issues 14

Severity Matrix 15

High Severity Issues 16

H-01 Buyers are mispriced on every native-coin purchase via hardcoded $1,700 price in _cryptoValues 16

H-02 Premium auto-compound positions can be permanently frozen after partialUnstake 18

Medium Severity Issues 19

M-01 Governance-eligible stakers earn the bonus APY on time before it was granted via settlement-time APY lookup in _effectiveAPYBpsForUserId 19

M-02 Premium stakers halve the early-exit penalty via near-total withdrawal in partialUnstake 21

M-03 Deployer can permanently lock ICO and Vesting admin functions via missing zero-address check on _owner in initialize 23

M-04 Future upgrades can corrupt Staking, ICO and Vesting state via unreserved storage in inherited base contracts 25

M-05 Owner cannot rescue surplus LRY because _accrueStakeFor counts each compounded reward twice in _totalObligated 27

M-06 Stakers lose or gain rewards they already earned because _refreshMultipliersOfUserId changes the whale multiplier before accruing them 29

Low Severity Issues **31**

L-01 Anyone can revert batchClaimFor by front-running it with claimTokens for one entry 31

L-02 KYC signatures have no expiration and no amount in _verifyAndConsumeKycSignature 33

Informational Severity Issues **34**

I-01 Owner can lose admin control permanently through single-step transferOwnership and renounceOwnership 34

I-02 Enhanced stakers with the governance bonus earn above the tier maxAPY cap because effectiveAPY ignores it 35

I-03 Misspelled function name vestingCalulations in Vesting 36

I-04 Vesting registers 35M LRY less than the documented Ecosystem and Liquidity allocations in initialize 37

Functional Tests 38

Threat Model 47

Automated Tests 74

Closing Summary & Disclaimer 75

Executive Summary

Project Name	Lendary
Protocol Type	KYC-gated token sale, multi-category token vesting and tiered LRY staking with a non-transferable receipt token.
Overview	The reviewed system distributes and stakes the LRY token. LendaryICO sells a fixed 50,000,000 LRY allocation to KYC-approved buyers for native coin, USDT or USDC, with an optional per-user custom price. LendaryVesting holds 915,000,000 LRY across six categories (private sale, ecosystem and staking rewards, liquidity, team and founders, treasury and operations, advisors and partners) and releases each allocation in monthly slices. LendaryStaking locks LRY in three tiers (Base, Enhanced, Premium) with different lock-ups, cooldowns, APYs and early-exit penalties, mints a non-transferable stLRY receipt, supports optional monthly auto-compounding, and limits large holders through a position cap, a velocity cap, a whale reward multiplier and a large-exit cooldown, all aggregated per backend-issued userId. Every purchase and stake requires a signature from an off-chain KYC signer, and all three contracts accept meta-transactions and are deployed behind transparent upgradeable proxies.
Audit Scope	The scope of this audit was to analyze the Lendary smart contracts for quality, security, and correctness.
Source Code link	github.com/LendaryCode/LendarySmartContract/tree/master
Contracts in Scope	contracts/ICO.sol contracts/Vesting.sol contracts/Staking.sol contracts/libraries/StakingLibrary.sol contracts/stLRY.sol contracts/LRYToken.sol contracts/KycSignatureGate.sol contracts/utils/MetaTransaction.sol
Branch	main
Commit Hash	ca3bf3c
Language	Solidity

Blockchain	EVM
Method	Manual Analysis, Threat Modelling
Review 1	23rd September - 10th October, 2026
Updated Code Received	19th October, 2026
Review 2	23rd October, 2026
Fixed In	f5523187a7435dcecb026af2096b473df7115464

Number of Issues per Severity



Critical	0 (0%)
High	2 (14%)
Medium	6 (43%)
Low	2 (14%)
Informational	4 (29%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	**1**	0
Partially Resolved	0	0	0	0	0
Resolved	0	**2**	**6**	**1**	**4**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Buyers are mispriced on every native-coin purchase via hardcoded $1,700 price in _cryptoValues	High	Resolved
2	Premium auto-compound positions can be permanently frozen after partialUnstake	High	Resolved
3	Governance-eligible stakers earn the bonus APY on time before it was granted via settlement-time APY lookup in _effectiveAPYBpsForUserId	Medium	Resolved
4	Premium stakers halve the early-exit penalty via near-total withdrawal in partialUnstake	Medium	Resolved
5	Deployer can permanently lock ICO and Vesting admin functions via missing zero-address check on _owner in initialize	Medium	Resolved
6	Future upgrades can corrupt Staking, ICO and Vesting state via unreserved storage in inherited base contracts	Medium	Resolved
7	Owner cannot rescue surplus LRY because _accrueStakeFor counts each compounded reward twice in _totalObligated	Medium	Resolved

Issue No.	Issue Title	Severity	Status
8	Stakers lose or gain rewards they already earned because _refreshMultipliersOfUserId changes the whale multiplier before accruing them	Medium	Resolved
9	Anyone can revert batchClaimFor by front-running it with claimTokens for one entry	Low	Resolved
10	KYC signatures have no expiration and no amount in _verifyAndConsumeKycSignature	Low	Acknowledged
11	Owner can lose admin control permanently through single-step transferOwnership and renounceOwnership	Informational	Resolved
12	Enhanced stakers with the governance bonus earn above the tier maxAPY cap because effectiveAPY ignores it	Informational	Resolved
13	Misspelled function name vestingCalulations in Vesting	Informational	Resolved
14	Vesting registers 35M LRY less than the documented Ecosystem and Liquidity allocations in initialize	Informational	Resolved

Checked Vulnerabilities

- ✓ Access Management
- ✓ Arbitrary write to storage
- ✓ Centralization of control
- ✓ Ether theft
- ✓ Improper or missing events
- ✓ Logical issues and flaws
- ✓ Arithmetic Computations Correctness
- ✓ Race conditions/front running
- ✓ SWC Registry
- ✓ Re-entrancy
- ✓ Timestamp Dependence
- ✓ Gas Limit and Loops
- ✓ Exception Disorder
- ✓ Gasless Send
- ✓ Use of tx.origin
- ✓ Malicious libraries
- ✓ Compiler version not fixed
- ✓ Address hardcoded
- ✓ Divide before multiply
- ✓ Integer overflow/underflow
- ✓ ERC's conformance
- ✓ Dangerous strict equalities
- ✓ Tautology or contradiction
- ✓ Return values of low-level calls
- ✓ Missing Zero Address Validation
- ✓ Private modifier
- ✓ Revert/require functions
- ✓ Multiple Sends
- ✓ Using suicide
- ✓ Using delegatecall

- ✓ Upgradeable safety
- ✓ Style guide violation
- ✓ Using throw
- ✓ Unsafe type inference
- ✓ Using inline assembly
- ✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

H-01 Buyers are mispriced on every native-coin purchase via hardcoded $1,700 price in _cryptoValues

Resolved

Path

`contracts/ICO.sol`

Function Name

`_cryptoValues(), calculateTokens(), buyTokens()`

Description

`ICO` sells LRY across time-bound phases at a fixed USD price per token. Phase 1 is set up in `initialize` at a price of 0.075 USD and a limit of 50,000,000 LRY. A KYC-approved buyer calls `buyTokens` and picks a payment type: 1 for the chain's native coin (the NatSpec says BNB/ETH), 2 for USDT, 3 for USDC. For native payments, `buyTokens` replaces `_buyAmount` with `msg.value` and calls `calculateTokens`. That function gets a USD rate and a decimal divisor from `_cryptoValues`, multiplies the payment by the rate, and divides by the divisor to get `_amountGivenInUsd`. `_calculateTokensInternal` then divides that USD figure by the phase price, or by the buyer's `userCustomPrice` override if one is set, to decide how many LRY to send. `buyTokens` adds the result to `totalTokenSold` and the USD figure to `totalUSDRaised`, forwards the native coin to `receiverAddress`, and transfers the LRY to the buyer straight away.

The contract stores an `ETHOracle` reference, set in `initialize`, and the `_cryptoValues` NatSpec says it returns an oracle price. However, for payment type 1, `_cryptoValues` never reads the oracle. It returns a fixed constant of 1,700 USD (scaled by 10^8) for one whole native coin, and the oracle call is commented out. Stablecoin payments are handled correctly at a 1:1 USD rate. So every native-coin purchase is valued as if the native coin is always worth exactly 1,700 USD, whatever its real market price, on whatever chain the contract is deployed to. There is no way to correct this after deployment, because no setter updates the rate and no other code path reads `ETHOracle`.

H-01 Buyers are mispriced on every native-coin purchase via hardcoded $1,700 price in _cryptoValues *(continued)*

When the native coin trades below 1,700 USD, every native buyer gets more LRY than they paid for, and the protocol bears the shortfall. If the contract runs on BNB Chain, as the NatSpec suggests, and BNB trades at 600 USD, one BNB buys 22,666 LRY at the 0.075 USD phase price instead of the correct 8,000 LRY. That is about 2.83 times the fair amount. Any KYC-approved buyer can repeat this freely. Taking the full 50,000,000 LRY allocation, priced at 3,750,000 USD, would cost roughly 2,206 BNB, about 1,323,000 USD. The buyer then receives the tokens at once and can sell them below the sale price. The same applies on Ethereum whenever ETH drops below 1,700 USD. When the native coin trades above 1,700 USD, honest buyers lose instead. At an ETH price of 3,400 USD, one ETH buys 22,666 LRY rather than 45,333 LRY, so the buyer loses half the value they sent. In both cases `totalUSDRaised` and the `BuyTokenDetail` event report USD amounts that do not match the value actually received.

So no native-coin purchase is ever priced at market. Depending on which way the price has moved, either the protocol sells its token allocation at a deep discount or buyers overpay for the tokens they receive.

Impact High

When the native coin trades below 1,700 USD, buyers can drain the sale allocation for a fraction of its value. With BNB at 600 USD, the full 50,000,000 LRY allocation (3,750,000 USD at the phase price) goes for about 1,323,000 USD. The protocol permanently loses about 2,427,000 USD of sale proceeds, and the buyer captures that difference as profit. When the native coin trades above 1,700 USD, every native-coin buyer loses value directly, for example 50% of each payment at an ETH price of 3,400 USD. Every native-coin sale also corrupts the protocol's `totalUSDRaised` accounting.

Likelihood Medium

Recommendation

Price native-coin purchases from a live oracle feed for the correct asset on the deployment chain, and validate that the answer is positive, fresh and complete before using it.

High Severity Issues

H-02 Premium auto-compound positions can be permanently frozen after partialUnstake

Resolved

Path

`Staking.sol`

Function Name

`_accrueStakeFor()`

Description

`_accrueStakeFor` books rewards linearly and moves `lastAccrualTimestamp` without moving `lastMonthlySettledAt`, and the auto-compound loop is skipped while a `partialUnstake` slice is in cooldown. If that slice is withdrawn after the next month boundary, `lastAccrualTimestamp` lands past that boundary and every later accrual underflows on:

```
uint256 dtStep = nextMonthEnd - uint256(pos.lastAccrualTimestamp);
```

A `partialUnstake` made more than `(MONTH - tier cooldown)` after the last month boundary always triggers this, since the cooldown itself ends past the next boundary.

Impact High

`unstake`, `unstakeEarly`, `partialUnstake` and `claimMonthlyRewards` revert forever for the position, locking its remaining principal and rewards, and `pendingRewards()` reverts for the whole `userId`.

Likelihood Medium

Recommendation

Never let `dtStep` underflow and never move `lastAccrualTimestamp` backwards, in both `_accrueStakeFor` and `_simulateAutoCompoundForPosition`.

Medium Severity Issues

M-01 Governance-eligible stakers earn the bonus APY on time before it was granted via settlement-time APY lookup in _effectiveAPYBpsForUserId

Resolved

Path

`contracts/Staking.sol, contracts/libraries/StakingLibrary.sol`

Function Name

`_effectiveAPYBpsForUserId(), effectiveAPY(), _computeRewardForUserId(), _accrueStakeFor(), submitGovernanceEligible()`

Description

`Staking` rewards governance participation with a temporary APY boost. The owner calls `submitGovernanceEligible` with a list of backend `userId`s, and for each one `governanceBonusUntil` is set to the current time plus 90 days. During that window every position of the `userId` should earn the tier's base APY plus `GOV_BONUS_BPS`, which is 1%. Outside the window it should earn the base APY only. The contract stores just the end timestamp. It does not record when the grant started.

Rewards are not streamed. They are settled lazily when `_accrueStakeFor` runs for a position, which happens through `unstake`, `unstakeEarly`, `partialUnstake`, `withdraw`, `accelerateCooldown` and `claimMonthlyRewards`. At settlement, `_computeRewardForUserId` charges simple interest on principal for the whole interval since `lastAccrualTimestamp`. It gets the rate from `_effectiveAPYBpsForUserId`, which calls `StakingLibrary.effectiveAPY`. That function only checks whether the current `block.timestamp` is earlier than `governanceBonusUntil` and, if so, adds the 1% bonus. It never compares the bonus window with the interval being settled. So whether the bonus applies is decided once, at the moment of settlement, and then used for the entire elapsed interval. That interval can go back months before the grant. The monthly steps inside the auto-compound loop of `_accrueStakeFor` have the same problem: each step is priced against the current timestamp, not the period it covers. The view functions `pendingRewards` and `pendingRewardsForStakeIds` go through the same path and show the same inflated figures.

M-01 Governance-eligible stakers earn the bonus APY on time before it was granted via settlement-time APY lookup in _effectiveAPYBpsForUserId *(continued)*

Assume a total supply of 1,000,000,000 LRY and a user with one 10,000,000 LRY Premium position (1% of supply, so the whale multiplier is 1.0x) earning the 12% base APY. The position was last accrued 300 days ago. The owner then grants the user governance eligibility. Within the next 90 days, the user calls any function that accrues the position, for example `claimMonthlyRewards`. The full 300 days are priced at 13% instead of 12%, which credits 82,191 LRY of bonus for time before the grant existed. The real 90 day bonus on this position is worth only 24,657 LRY, so the user receives more than three times the intended amount. The longer a position goes without accruing before a grant, the larger the windfall. A staker who expects to be granted eligibility can deliberately avoid touching their positions to maximise it.

The reverse also happens. If a granted user does not accrue the position before `governanceBonusUntil` passes, the next settlement prices the whole interval at the base APY. The bonus they earned during the 90 day window disappears completely. The same position above would lose the full 24,657 LRY.

As a result, the bonus a staker receives depends on when they happen to settle, not on whether they were eligible during the time being paid for. Early settlers are overpaid from the shared reward pool and late settlers are underpaid.

Impact Medium

A granted staker who settles inside the bonus window collects the 1% bonus over their entire unsettled history. In the example above that is 82,191 LRY instead of the intended 24,657 LRY, an overpayment of 57,534 LRY taken from the reward pool funded through `depositRewards`. A granted staker who settles only after the window ends loses the whole bonus they earned. Principal is not affected. The loss or windfall is limited to yield and scales with position size and the length of the unsettled interval.

Likelihood Medium

Recommendation

Apply the governance bonus only to the part of each accrual interval that overlaps the bonus window, for example by tracking the grant start time or by settling all of the user's positions when a grant is made or ends.

Medium Severity Issues

M-02 Premium stakers halve the early-exit penalty via near-total withdrawal in partialUnstake

Resolved

Path

`contracts/Staking.sol, contracts/libraries/StakingLibrary.sol`

Function Name

`partialUnstake(), unstakeEarly()`

Description

`Staking` offers Premium stakers two ways out before the lock-up ends. `unstakeEarly` closes the whole position. It charges the tier's `earlyExitPenalty`, which `initialize` sets to 5% for Premium, forfeits every accrued reward, marks the position `earlyExited`, and moves the remaining 95% into cooldown. `partialUnstake` is meant as a lighter option for trimming a position while keeping the rest locked, and it becomes available 90 days after `startTime` (`PARTIAL_UNSTAKE_EARLIEST`). It charges `PARTIAL_UNSTAKE_PENALTY_BPS`, which is 2.5%, on the slice being withdrawn. It forfeits only that slice's proportional share of `rewardAccrued`, sends the forfeited reward to `insuranceReserve`, and moves the rest of the slice into cooldown. The remaining principal stays staked and can be unstaked normally once the lock ends.

The only limit `partialUnstake` puts on the slice size is that `amount` must be greater than zero and strictly less than `pos.principal`. Nothing sets a maximum fraction, and nothing requires the remainder to stay at or above `minStake` or any other meaningful size. A Premium staker can therefore pass an amount equal to the full principal minus one wei. The contract treats that as a partial exit, charges 2.5% instead of 5%, and leaves a one-wei position behind. In economic terms that is a full early exit. `withdraw` then pays out the cooldown slice, and the one-wei remainder either sits there or is unstaked for free once the lock ends.

Take a Premium staker with 1,000,000 LRY who wants out 100 days into a 12 month lock. Through `unstakeEarly` they pay a 50,000 LRY penalty to `penaltyReceiver` and get 950,000 LRY back after the cooldown. If they instead call `partialUnstake` for 1,000,000 LRY minus one wei, the penalty is about 25,000 LRY and about 975,000 LRY enters cooldown. Almost all of the accrued reward is forfeited either way, because the forfeited share is proportional and the slice is nearly 100% of principal. The cooldown length is also the same, since both paths use the tier cooldown and neither qualifies for acceleration. The only real difference is that the staker keeps 25,000 LRY of penalty that the protocol should have collected. Any Premium staker past day 90 can do this with no special setup.

M-02 Premium stakers halve the early-exit penalty via near-total withdrawal in partialUnstake *(continued)*

So the 5% Premium early-exit penalty becomes a 2.5% penalty for every exit after day 90. That undercuts the lock-up commitment Premium's higher APY is meant to pay for, and diverts half of the expected penalty revenue away from `penaltyReceiver`.

Impact Medium

Every Premium staker who exits early after day 90 can avoid half the intended early-exit penalty, which is 2.5% of their principal (25,000 LRY on a 1,000,000 LRY position). The staker keeps that amount and the protocol's `penaltyReceiver` loses it. User principal is never at risk. The loss is penalty revenue only, and it repeats for every Premium early exit.

Likelihood Medium

Recommendation

Require the remainder after `partialUnstake` to stay above a meaningful minimum, such as `minStake`, or cap the share of principal a single partial unstake can remove, so that a near-total exit has to go through `unstakeEarly`.

Medium Severity Issues

M-03 Deployer can permanently lock ICO and Vesting admin functions via missing zero-address check on _owner in initialize

Resolved

Path

`contracts/ICO.sol, contracts/Vesting.sol`

Function Name

`initialize()`

Description

`LendaryICO` and `LendaryVesting` are deployed behind proxies, so their real setup happens in `initialize` rather than in the constructor. Each constructor runs `Ownable(_msgSender())` and `_disableInitializers()`, but that only affects the implementation contract's own storage. The proxy starts with no owner. It gets its owner only when `initialize` ends by calling `_transferOwnership(_owner)`. Every privileged action in both contracts uses the `onlyOwner` modifier. In `LendaryICO` these are `sendLeftoverTokens`, `updateReceiverAddress`, `setKycSigner` and `setUserPrice`. In `LendaryVesting` they are `registerPrivateSaleUser`, `sendLeftoverTokens`, `updateReceiverAddress` and `setKycSigner`.

Neither `initialize` checks that `_owner` is a non-zero address. The OpenZeppelin `Ownable` used here only rejects the zero address in its constructor and in the public `transferOwnership`. The internal `_transferOwnership` that `initialize` calls accepts address zero without complaint. So if the deployment script passes a zero `_owner`, through an unset environment variable or a wrong argument order, initialization succeeds and the proxy ends up with `owner()` equal to address zero. The `initializer` modifier blocks a second call to `initialize`, and nobody can call `transferOwnership` because no one is the owner. The `Staking` contract shows the intended pattern: its `initialize` rejects a zero `_admin` with `ZeroAdmin`. The other two contracts do not.

After a bad initialization, every owner function in both contracts is unreachable. In `LendaryICO`, unsold LRY from the 50,000,000 LRY sale allocation cannot be recovered with `sendLeftoverTokens`. The hardcoded `receiverAddress` can never be changed. Per-user prices cannot be set. A compromised KYC signer cannot be rotated, so whoever holds that key could keep approving purchases indefinitely. In `LendaryVesting`, `registerPrivateSaleUser` can never run, so the private sale cannot onboard anyone. Unallocated private-sale LRY is stuck, and the treasury address and KYC signer are frozen as well.

M-03 Deployer can permanently lock ICO and Vesting admin functions via missing zero-address check on _owner in initialize *(continued)*

So a single bad constructor argument leaves both contracts without an admin. The only way to recover is a proxy upgrade that adds a new initializer, and that is possible only if the proxy admin was set up correctly and separately.

Impact Medium

If `_owner` is set to zero, all admin functionality in `LendaryICO` and `LendaryVesting` is lost. Unsold ICO tokens (up to the full 50,000,000 LRY allocation) and unallocated private-sale tokens stay locked in the contracts. The private sale cannot run, and the KYC signer and treasury address cannot be changed. The situation can only be fixed through a proxy upgrade by the proxy admin.

Likelihood Medium

Recommendation

Reject a zero `_owner` in `initialize` for both `LendaryICO` and `LendaryVesting`, the same way `Staking` validates its admin.

Medium Severity Issues

M-04 Future upgrades can corrupt Staking, ICO and Vesting state via unreserved storage in inherited base contracts

Resolved

Path

`Staking.sol, ICO.sol, Vesting.sol, KycSignatureGate.sol, MetaTransaction.sol`

Function Name

`initialize()`

Description

`LendaryStaking`, `LendaryICO` and `LendaryVesting` are built to sit behind `LendaryTransparentUpgradeableProxy`. Each has an initializer-guarded `initialize` and disables initializers in its constructor. Behind a transparent proxy, all state lives in the proxy's storage, and every new implementation must read the existing slots exactly the way the previous one wrote them. Solidity lays out state variables one after another in inheritance order: base contracts first, from the leftmost parent, then the child's own variables. For example, `LendaryStaking` inherits `Ownable`, `Pausable`, `ReentrancyGuard`, `Initializable`, `KycSignatureGate` and `BasicMetaTransaction`, in that order, and only then declares its own state. `LendaryICO` and `LendaryVesting` follow the same pattern with `Ownable`, `ReentrancyGuard`, `KycSignatureGate`, `Initializable` and `BasicMetaTransaction`.

None of the parent contracts reserve space for future growth. `KycSignatureGate` declares `kycSigner`, `kycNonces` and `_isSignatureUsed` in plain sequential slots, with no `__gap` array and no namespaced (ERC-7201) storage. `BasicMetaTransaction` does the same with its private `nonces` mapping. All three contracts also inherit the standard, non-upgradeable OpenZeppelin `Ownable`, `Pausable` and `ReentrancyGuard` rather than the upgradeable variants. Those standard versions are not meant to keep a stable layout across library upgrades, and moving to the upgradeable variants later would change the layout too. So the slot position of every child variable depends on the exact variable count of every parent at the time of deployment.

M-04 Future upgrades can corrupt Staking, ICO and Vesting state via unreserved storage in inherited base contracts *(continued)*

In practice, if a later version adds a single variable to `KycSignatureGate`, for example a signer expiry or a second signer, every variable that comes after it moves by one slot. In `LendaryStaking`, that includes the `stakes` mapping, `_totalStakedByUserId`, `_totalObligated`, `tierConfig`, `monthlyAutoCompoundEnabled` and `governanceBonusUntil`. In `LendaryICO` it includes `phaseInfo`, `totalTokenSold` and `receiverAddress`. The new implementation would read the old values from the wrong slots. Staked principals, reward obligations, tier configuration and sale accounting would decode as unrelated numbers or addresses, and the next state-changing call would write that corrupted state back permanently. The same thing happens if a parent is reordered or removed, if a child variable is inserted anywhere except at the end, or if the OpenZeppelin dependency is upgraded and changes the layout of `Ownable`, `Pausable` or `ReentrancyGuard`.

The contracts therefore have no structural protection against a routine-looking change to a shared base contract. A single upgrade that touches the inheritance chain can silently corrupt the state that tracks every staker's principal and rewards and every buyer's sale accounting.

Impact Medium

An incompatible upgrade would corrupt live storage in whichever contract is upgraded. In `LendaryStaking` that means every position's principal, `rewardAccrued` and timestamps, plus the protocol's obligation accounting. This can lead to wrong payouts, failed withdrawals or locked staked LRY, and it cannot be undone once the corrupted state is written. The risk only becomes real when an upgrade changes the inheritance chain or the order of variables.

Likelihood Low

Recommendation

Reserve storage in every custom base contract, using `__gap` arrays or ERC-7201 namespaced storage, inherit the OpenZeppelin upgradeable variants, and validate the storage layout with upgrade-safety tooling before every upgrade.

Medium Severity Issues

M-05 Owner cannot rescue surplus LRY because _accrueStakeFor counts each compounded reward twice in _totalObligated

Resolved

Path

`contracts/Staking.sol`

Function Name

`_accrueStakeFor, _adjustObligated, rescueERC20`

Description

`Staking` holds two kinds of LRY in one balance: what it owes stakers (active principal, cooldown principal and accrued rewards) and spare reward funding the owner adds through `depositRewards`. The private counter `_totalObligated` tracks the first kind. `_openStakePosition` adds the staked amount, accruing rewards adds the reward, and payouts, penalties and fees subtract what leaves the contract, all through `_adjustObligated`. `rescueERC20` relies on this counter. When the owner tries to pull LRY out, it reverts with `WouldUnderfund` unless the contract's LRY balance stays at or above `_totalObligated` plus the amount being rescued. The design rule is that `_totalObligated` always equals what stakers are actually owed.

When monthly auto-compound is on for a position, `_accrueStakeFor` handles each monthly step in two stages. First it computes `stepReward` for the interval up to the month boundary, adds it to `pos.rewardAccrued`, and calls `_adjustObligated` with plus `stepReward`. That correctly records the new liability. Then it takes the whole of `pos.rewardAccrued` as `r`, which includes the `stepReward` just recorded and any earlier accrued reward that was also recorded when it accrued. It moves `r` into `pos.principal` and resets `rewardAccrued` to zero. But it also calls `_adjustObligated` again with plus `r`. Moving value from `rewardAccrued` into principal does not change how much the contract owes; it only reclassifies a liability that is already counted. Adding `r` a second time means every compounded token is counted twice. Exits only subtract what is actually paid: `_settleCooldownExit` subtracts `netPrincipal + reward`, and penalties and forfeits are subtracted when they are sent out. So the extra copy of each compounded reward is never removed and builds up permanently in `_totalObligated`.

M-05 Owner cannot rescue surplus LRY because _accrueStakeFor counts each compounded reward twice in _totalObligated *(continued)*

For example, a single Premium position of 10,000,000 LRY at 12% APY with auto-compound on compounds 11 times over a 12 month lock and folds in about 1,156,000 LRY of rewards. `_totalObligated` is overstated by that same 1,156,000 LRY. Even after the staker unstakes and withdraws everything, the counter still reports 1,156,000 LRY owed to no one. Across all auto-compounding positions, the overstatement grows with every monthly step. Any spare reward funding up to that amount is then blocked by `rescueERC20`. Once the overstatement exceeds the spare balance, the owner cannot rescue any LRY at all, even when nothing is owed to stakers.

So the contract's own record of what it owes no longer matches reality. The safety check meant to protect staker funds instead traps the protocol's surplus LRY with no way to correct the counter.

Impact Medium

Protocol-owned LRY equal to the total of all compounded rewards becomes impossible to recover through `rescueERC20`, for example about 1,156,000 LRY for a single 10,000,000 LRY Premium position compounding over one year. The lockup is permanent because `_totalObligated` can only be changed by real flows and has no admin correction. Staker funds are not at risk, because subtractions never exceed real liabilities, so user payouts never underflow.

Likelihood Medium

Recommendation

Do not adjust `_totalObligated` when moving accrued rewards into principal during auto-compound, since the liability was already recorded when the reward accrued.

Medium Severity Issues

M-06 Stakers lose or gain rewards they already earned because _refreshMultipliersOfUserId changes the whale multiplier before accruing them

Resolved

Path

`contracts/Staking.sol, contracts/libraries/StakingLibrary.sol`

Function Name

`_refreshMultipliersOfUserId(), _openStakePosition(), _settleCooldownExit(), _accrueStakeFor(), partialUnstake()`

Description

`Staking` scales each position's reward rate with an anti-whale multiplier based on how much the owning `userId` has staked across all its positions and linked wallets. `StakingLibrary.computeWhaleMultiplier` returns 1.0x while `_totalStakedByUserId` is at or below 1% of LRY total supply. Between 1% and 2% it falls linearly to 0.7x, and above 2% it stays at 0.7x. Each position stores its own copy in `whaleMultiplierBps`. Rewards are not streamed. A position accrues lazily, only when `_accrueStakeFor` runs for that stake id. It then charges simple interest from `lastAccrualTimestamp` to now on principal, multiplied by the APY and by whatever `whaleMultiplierBps` holds at that moment. A user action only calls `_accrueStakeFor` for the one stake id being acted on (`stake` calls it for none), so a user's other positions can go unaccrued for months.

Every action that changes the aggregate calls `_refreshMultipliersOfUserId` afterwards. This happens when `_openStakePosition` opens a new stake, when `_settleCooldownExit` pays out a withdrawal or accelerated exit, when `partialUnstake` or `unstakeEarly` burns a penalty, and after each compounding step inside `_accrueStakeFor`. `_refreshMultipliersOfUserId` loops over every position of the `userId` that still has principal and overwrites its `whaleMultiplierBps` with the newly computed value. It does not accrue any of those positions first. Their `lastAccrualTimestamp` stays where it was. So the next time each sibling position accrues, the whole interval since its last checkpoint is priced at the new multiplier, even though the aggregate during most of that interval justified a different rate. A change that should only affect future yield is applied backwards to yield already earned.

M-06 Stakers lose or gain rewards they already earned because _refreshMultipliersOfUserId changes the whale multiplier before accruing them *(continued)*

Here is how an ordinary staker loses out. Assume a total supply of 1,000,000,000 LRY and two Premium positions for one `userId` totalling 10,000,000 LRY, or 1% of supply, so both run at 1.0x. The user leaves them alone for 180 days. At the 12% Premium APY they have earned 591,780 LRY, but none of it is checkpointed. The user then opens a third position of 5,000,000 LRY, which passes both the velocity cap and the 2% position cap. The aggregate rises to 15,000,000 LRY and `_refreshMultipliersOfUserId` sets both older positions to 0.85x. When they next accrue, the 180 days already earned are paid at 0.85x, which is 503,013 LRY. The user silently loses 88,767 LRY that they earned before the new stake existed.

A staker can also use this on purpose to gain. Take a user whose aggregate is 15,000,000 LRY (0.85x), made up of 10,000,000 LRY in long-lock Premium positions and a 5,000,000 LRY Base position with a shorter lock. The user never touches the Premium positions. Once the Base lock ends, the user calls `unstake` on it and then `withdraw`. `_settleCooldownExit` lowers the aggregate to 10,000,000 LRY, and `_refreshMultipliersOfUserId` sets the untouched Premium positions back to 1.0x. If they went 180 days without accruing, every one of those days is now paid at 1.0x instead of the 0.85x that applied. The user takes 88,767 LRY more than the protocol's rules allow, out of the reward pool shared with other stakers. A smaller version of this happens whenever a `partialUnstake` or `unstakeEarly` penalty lowers the aggregate.

In short, rewards depend on the order and timing of a user's actions rather than on how much they actually staked over time. Honest multi-position stakers lose yield they already earned, and anyone who plans their exits can push the anti-whale discount off their past yield.

Impact Medium

Stakers with several positions whose aggregate sits between 1% and 2% of supply lose earned rewards when they add stake. In the example above that is 88,767 LRY, or 15% of 180 days of yield on 10,000,000 LRY. A user who sequences exits deliberately can claim the same amount in excess rewards from the shared reward pool. Principal is never at risk. The damage is limited to mispriced yield and scales with position size, time left unaccrued, and the size of the multiplier change, up to the full 30% band.

Likelihood Medium

Recommendation

Accrue all active positions of a `userId` at their current multiplier before `_refreshMultipliersOfUserId` changes it, so that each new multiplier only applies to rewards earned from that point on.

Low Severity Issues

L-01 Anyone can revert batchClaimFor by front-running it with claimTokens for one entry

Resolved

Path

`contracts/Vesting.sol`

Function Name

`batchClaimFor(), claimTokens()`

Description

`Vesting` releases LRY to registered beneficiaries in scheduled periods per vesting category. A beneficiary's claimable amount for a category comes from `tokensToBeClaimed`, which compares the claim period the schedule has reached with the periods already claimed. Anyone can call `claimTokens` for any beneficiary and category. It takes no signature and no access control, and the tokens always go to the beneficiary's address. To save gas and operations, `batchClaimFor` lets a caller, normally a backend keeper, process up to 25 beneficiary and category pairs in one transaction. It checks the array lengths and then calls the shared internal `_claimForInternal` for each pair in turn.

`_claimForInternal` is strict. It reverts with `UserNotRegistered` if the beneficiary has no allocation in that category. It then reads the claimable amount from `tokensToBeClaimed`, and if that amount is zero it reverts with `AllTokensClaimed` or `NothingToClaim`. Only when the amount is non-zero does it update `claimedAmount` and the claim count and transfer the tokens. `batchClaimFor` does not catch or skip these reverts, so one entry with nothing to claim reverts the whole transaction and rolls back every other claim in the batch. The function's documentation tells callers to pre-check each entry with `tokensToBeClaimed` before building the batch. That check happens off-chain when the batch is built, and nothing stops the state from changing before the batch is mined.

An attacker watching the mempool sees a keeper's `batchClaimFor` covering 25 beneficiaries and front-runs it with a `claimTokens` call for any one of those beneficiaries and categories. The attacker's call pays that beneficiary their current period and advances their claim count. When the keeper's batch executes, that entry now has a claimable amount of zero, `_claimForInternal` reverts with `NothingToClaim`, and the other 24 claims are undone. The keeper pays gas for the failed batch, has to rebuild it and resubmit it, and the attacker can do the same against the next submission. No attacker is needed either: if any beneficiary in the batch calls `claimTokens` for themselves shortly before the batch is mined, the batch fails the same way.

L-01 Anyone can revert batchClaimFor by front-running it with claimTokens for one entry *(continued)*

As a result, batch claiming is unreliable and anyone can grief it repeatedly for the cost of a single claim, which delays vesting payouts for every other beneficiary in the batch.

Impact Low

No tokens are lost or redirected, because every claim still pays the rightful beneficiary. The keeper loses the gas of each reverted batch, and up to 24 other beneficiaries have their payout delayed every time the attack is repeated. The attacker makes no profit and pays the gas for each front-running claim.

Likelihood Medium

Recommendation

Make `batchClaimFor` skip entries that have nothing to claim instead of reverting, so one already-claimed entry cannot cancel the rest of the batch.

Low Severity Issues

L-02 KYC signatures have no expiration and no amount in _verifyAndConsumeKycSignature

Acknowledged

Path

`contracts/KycSignatureGate.sol`

Function Name

`_verifyAndConsumeKycSignature()`

Description

`stake` in `Staking` and `buyTokens` in `ICO` both require a backend KYC signature, which `_verifyAndConsumeKycSignature` checks. The signed message covers only the `userId`, the wallet, the wallet's current entry in `kycNonces`, the chain id and the contract address. It has no expiration timestamp and no amount.

Because there is no expiration, a signature stays valid until it is used, no matter how much time has passed. A user can get a signature, hold it, and use it weeks or months later, even after the backend would no longer approve them. Because there is no amount, the same signature works for a stake or purchase of any size, so the backend cannot limit what it approves.

Impact Low

An old or no longer intended signature can still be used to stake or buy any amount within the on-chain limits. No funds are lost directly. The backend's only way to cancel an unused signature is to rotate `kycSigner`, which cancels every user's outstanding signatures at once.

Likelihood Medium

Recommendation

Add an expiration timestamp and the approved amount to the signed message, and check both on-chain.

Informational Severity Issues

I-01 Owner can lose admin control permanently through single-step transferOwnership and renounceOwnership

Resolved

Path

```
contracts/Staking.sol, contracts/Vesting.sol, contracts/ICO.sol,
contracts/LRYToken.sol, contracts/stLRY.sol
```

Function Name

```
transferOwnership(), renounceOwnership()
```

Description

`Staking`, `Vesting`, `ICO` and `LRYToken` inherit OpenZeppelin `Ownable`. There, `transferOwnership` moves ownership to the new address immediately, without the new owner having to accept it. If the owner types a wrong or unreachable address, ownership is lost for good. None of the five contracts overrides `renounceOwnership`, so one accidental call sets the owner to the zero address. That includes `stLRY`, which already uses the two-step `Ownable2Step`.

Losing ownership permanently disables every `onlyOwner` function: pausing, rotating the KYC signer, depositing staking rewards, toggling auto-compound, updating receivers and rescuing tokens.

Impact Low

An owner mistake would permanently remove admin control over the protocol, including emergency pause. No user can exploit this.

Likelihood Low

Recommendation

Use `Ownable2Step` in all contracts and override `renounceOwnership` to revert.

Informational Severity Issues

I-02 Enhanced stakers with the governance bonus earn above the tier maxAPY cap because effectiveAPY ignores it

Resolved

Path

`contracts/libraries/StakingLibrary.sol, contracts/Staking.sol`

Function Name

`effectiveAPY(), _effectiveAPYBpsForUserId()`

Description

Each tier in `tierConfig` stores a `baseAPY` and a `maxAPY`, and `maxAPY` is documented as the maximum, governance-boosted APY for that tier. `_effectiveAPYBpsForUserId` passes only `baseAPY` to `effectiveAPY`. That function adds the 1% `GOV_BONUS_BPS` when the user's governance bonus is active and returns the sum without comparing it with `maxAPY`. `maxAPY` is never read anywhere in the contracts.

With the configured values, Base reaches 8% + 1% = 9%, exactly its cap. Premium reaches 12% + 1% = 13%, below its 13.6% cap. Enhanced reaches 9.6% + 1% = 10.6%, which is above its 10.4% cap. Every Enhanced position whose `userId` has the governance bonus therefore earns more than the tier's intended maximum.

Impact Low

Enhanced stakers with an active governance bonus earn 0.2% APY more than the documented cap, paid from the protocol's reward funds. The overpayment is small and stops when the 90 day bonus expires.

Likelihood Low

Recommendation

Cap the effective APY at the tier's `maxAPY` in `effectiveAPY`.

Informational Severity Issues

I-03 Misspelled function name vestingCalulations in Vesting

Resolved

Path

```
contracts/Vesting.sol
```

Function Name

```
vestingCalulations()
```

Description

`Vesting` declares a function named `vestingCalulations`. It sums the release percentages between a user's last claim period and the current one, and `tokensToBeClaimed` calls it to work out the claimable amount. The name is misspelled and should be `vestingCalculations`. The function works correctly, but the misspelling makes the code harder to read and search.

Impact Low

There is no effect on funds or behaviour. It only affects readability and maintainability.

Likelihood Low

Recommendation

Rename the function to `vestingCalculations` and update every call site.

Informational Severity Issues

I-04 Vesting registers 35M LRY less than the documented Ecosystem and Liquidity allocations in initialize

Resolved

Path

`contracts/Vesting.sol, contracts/LRYToken.sol`

Function Name

`initialize()`

Description

The NatSpec in `LRYToken` and `Vesting` documents the Ecosystem & Staking Rewards allocation as 25% of supply (250,000,000 LRY) and the Liquidity allocation as 10% of supply (100,000,000 LRY). However, `initialize` in `Vesting` registers only 225,000,000 LRY for category 2 (Ecosystem & Staking Rewards) and 90,000,000 LRY for category 3 (Liquidity). The vesting percentages for both categories add up to 100% of the registered amount, so the vesting schedule never releases the remaining 25,000,000 and 10,000,000 LRY.

Each shortfall is exactly 10% of its documented allocation. That matches the 10% immediate release `Vesting` applies to private sale purchases, which suggests a TGE unlock handled outside the contract. Nothing in the code or comments says so, and no on-chain logic accounts for the missing 35,000,000 LRY.

Impact Low

On-chain vesting doesn't match the published tokenomics. Users and integrators can't verify from the contract where the 35,000,000 LRY goes, or whether it's released at TGE, kept by the owner, or left unallocated.

Likelihood Low

Recommendation

Make the registered amounts match the documented allocations, or document the 10% TGE unlock and how it is distributed.

Functional Tests

Some of the tests performed are mentioned below:

KycSignatureGate

- ✔ _setKycSigner reverts if the signer address is zero
- ✔ _setKycSigner stores the new signer
- ✔ a KYC check reverts while no signer is set
- ✔ a KYC check reverts if the signature is not from the KYC signer
- ✔ a KYC check reverts if the same signature is used twice
- ✔ the signed message binds userId, wallet, nonce, chain id and contract
- ✔ a signature issued for one wallet fails for any other wallet
- ✔ a signature issued for one contract fails on the other contracts
- ✔ each accepted signature increases the wallet's KYC nonce by one
- ✔ ECDSA recovery rejects malleable high-s signatures
- ✔ kycSignDigest returns the digest the signer must sign for the current nonce

BasicMetaTransaction

- ✔ executeMetaTransaction reverts if the signer is not the user
- ✔ executeMetaTransaction reverts if the signature recovers to the zero address
- ✔ executeMetaTransaction increases the user's nonce before the call
- ✔ executeMetaTransaction runs the call as the user by appending their address
- ✔ executeMetaTransaction reverts if the relayed call fails
- ✔ executeMetaTransaction emits MetaTransactionExecuted with the user and relayer
- ✔ a replayed meta-transaction fails because the nonce has moved on
- ✔ the signed hash binds the nonce, contract, chain id and call data
- ✔ _msgSender returns msg.sender for direct calls
- ✔ getNonce returns the user's current nonce
- ✔ getChainID returns the current chain id

LRYToken

- ✅ constructor mints 1,000,000,000 LRY to the initial owner
- ✅ constructor sets the initial owner as the contract owner
- ✅ name is LRYToken, symbol is LRY and decimals is 18
- ✅ total supply equals the sum of all balances
- ✅ transfer moves tokens between accounts
- ✅ transferFrom moves tokens and spends the allowance
- ✅ burn lowers the caller's balance and the total supply
- ✅ burnFrom reverts without enough allowance
- ✅ burnFrom burns another account's tokens and spends the allowance
- ✅ no function can mint new tokens after deployment
- ✅ the owner has no special power over balances or supply

stLRY

- ✅ constructor reverts if the admin address is zero
- ✅ name is Staked LRY and symbol is stLRY
- ✅ setStakingContract is owner only
- ✅ setStakingContract reverts if the address is zero
- ✅ setStakingContract reverts once the pointer is locked
- ✅ setStakingContract stores the pointer and emits StakingContractSet
- ✅ lockStakingContract is owner only
- ✅ lockStakingContract reverts if no staking contract is set
- ✅ lockStakingContract reverts if the pointer is already locked
- ✅ lockStakingContract locks the pointer and emits StakingContractLocked
- ✅ mint reverts for any caller other than the staking contract
- ✅ burn reverts for any caller other than the staking contract
- ✅ mint increases the receiver's balance and the total supply
- ✅ burn decreases the holder's balance and the total supply

- ✅ transfer always reverts
- ✅ transferFrom always reverts
- ✅ approve always reverts
- ✅ increaseAllowance and decreaseAllowance always revert
- ✅ _update blocks any movement that is not a mint or a burn
- ✅ ownership moves only after the new owner calls acceptOwnership

StakingLibrary

- ✅ computeWhaleMultiplier returns 1.0x at or below 1% of supply
- ✅ computeWhaleMultiplier returns 0.7x at or above 2% of supply
- ✅ computeWhaleMultiplier falls linearly between 1% and 2% of supply
- ✅ computeWhaleMultiplier returns 1.0x when supply is zero
- ✅ computeReward returns zero for zero time or zero principal
- ✅ computeReward scales with principal, APY, multiplier and time
- ✅ effectiveAPY adds 1% only while the governance bonus is active
- ✅ rewardAccrualCapTs stops reward accrual at lock end
- ✅ maxAutoCompoundMonthEndTs stops auto-compound one month before lock end
- ✅ isLargeExit is true only above 1% of supply
- ✅ cooldownFor returns the large-exit cooldown above 1% of supply
- ✅ cooldownFor returns the tier cooldown at or below 1% of supply
- ✅ accelerationFeeBps returns 0.5%, 1%, 1.5% or 2% by time remaining
- ✅ auto-compound needs the flag, opt-in, non-Base tier, no exit and no cooldown
- ✅ the storage and memory auto-compound checks give the same answer

LendaryStaking

- ✅ the implementation cannot be initialized directly
- ✅ initialize can only run once
- ✅ initialize reverts if the minimum stake is zero
- ✅ initialize reverts if the LRY token address is zero

- ✔ initialize reverts if the stLRY token address is zero
- ✔ initialize reverts if the KYC signer address is zero
- ✔ initialize reverts if the admin address is zero
- ✔ initialize reverts if the penalty receiver address is zero
- ✔ initialize reverts if the insurance reserve address is zero
- ✔ initialize writes the Base, Enhanced and Premium tier configurations
- ✔ initialize transfers ownership to the admin
- ✔ stake reverts while the contract is paused
- ✔ stake reverts on an empty userId
- ✔ stake reverts below the minimum stake
- ✔ stake reverts on Base tier with auto-compound
- ✔ stake reverts without a valid KYC signature
- ✔ stake links the wallet to the userId on first use
- ✔ stake reverts if the wallet is linked to another userId
- ✔ stake reverts if the userId would hold more than 2% of supply
- ✔ stake reverts if new stake in the window exceeds 0.5% of supply
- ✔ stake pulls the LRY from the staker
- ✔ stake mints stLRY 1:1 to the staker
- ✔ stake opens a new position numbered 1, 2, 3 per userId
- ✔ stake sets the lock end from the tier lock-up
- ✔ stake refreshes the whale multiplier of every position of the userId
- ✔ stake emits Staked with the tier, amount and auto-compound flag
- ✔ position actions revert for a wallet not linked to a userId
- ✔ position actions revert for an unknown stake id
- ✔ active-position actions revert during cooldown or after an early exit
- ✔ unstake reverts before the lock ends
- ✔ unstake books pending rewards first
- ✔ unstake moves the whole principal into cooldown
- ✔ unstake uses the large-exit cooldown above 1% of supply
- ✔ unstake makes a standard exit eligible for cooldown acceleration

- ✔ unstakeEarly reverts once the lock has ended
- ✔ unstakeEarly takes the tier penalty of 3%, 4% or 5% of principal
- ✔ unstakeEarly forfeits all booked rewards
- ✔ unstakeEarly sends the penalty and forfeited rewards to the penalty receiver
- ✔ unstakeEarly burns stLRY for the penalty
- ✔ unstakeEarly moves the remaining principal into cooldown
- ✔ partialUnstake reverts for non-Premium positions
- ✔ partialUnstake reverts before 90 days
- ✔ partialUnstake reverts on a zero amount or the full principal
- ✔ partialUnstake takes a 2.5% penalty on the slice
- ✔ partialUnstake sends the slice's share of rewards to the insurance reserve
- ✔ partialUnstake keeps the remaining principal staked
- ✔ withdraw reverts if the position is not in cooldown
- ✔ withdraw reverts before the cooldown ends
- ✔ withdraw pays the cooldown principal and burns its stLRY
- ✔ withdraw pays booked rewards when the position is fully closed
- ✔ withdraw leaves rewards with a still-staked remainder
- ✔ withdraw cannot pay the same cooldown twice
- ✔ accelerateCooldown reverts if the position is not in cooldown
- ✔ accelerateCooldown reverts after an early, partial or large exit
- ✔ accelerateCooldown reverts once the cooldown has ended
- ✔ accelerateCooldown charges 0.5% to 2% depending on time remaining
- ✔ accelerateCooldown sends the fee to the insurance reserve
- ✔ accelerateCooldown pays net principal plus all booked rewards
- ✔ claimMonthlyRewards reverts before a month has passed since the last claim
- ✔ claimMonthlyRewards reverts before lock end while auto-compound is active
- ✔ claimMonthlyRewards reverts when no rewards are booked
- ✔ claimMonthlyRewards pays booked rewards and resets the monthly checkpoint
- ✔ rewards stop accruing at lock end
- ✔ auto-compound re-stakes each month except the last and mints stLRY

- ✅ auto-compound stops instead of crossing the 2% position cap
- ✅ the governance bonus adds 1% APY while active
- ✅ pause and unpause are owner only
- ✅ setKycSigner is owner only and rejects the zero address
- ✅ setKycSigner emits KycSignerUpdated
- ✅ setMinStake is owner only and rejects zero
- ✅ updatePenaltyReceiver is owner only and rejects the zero address
- ✅ updateInsuranceReserve is owner only and rejects the zero address
- ✅ submitGovernanceEligible is owner only and rejects an empty userId
- ✅ submitGovernanceEligible grants the bonus for 90 days
- ✅ setMonthlyAutoCompoundEnabled is owner only and emits the toggle event
- ✅ depositRewards is owner only and rejects zero
- ✅ depositRewards pulls LRY from the owner
- ✅ rescueERC20 is owner only and rejects the zero recipient
- ✅ rescueERC20 cannot take LRY below staker obligations
- ✅ rescueERC20 can take any other token
- ✅ getWalletsByUserId returns every wallet linked to a userId
- ✅ getStakeIdsByWallet returns the positions a wallet opened
- ✅ getStakeIdsByUserId returns every position of a userId
- ✅ aggregatedStakedOf returns principal plus cooldown for a userId
- ✅ totalStakedByWallet returns zero for an unlinked wallet
- ✅ effectiveAPYBps includes the governance bonus while active
- ✅ currentWhaleMultiplierBps matches the userId's aggregate stake
- ✅ pendingRewards returns zero for an empty userId
- ✅ pendingRewardsForStakeIds returns zero for unknown stake ids

LendaryICO

- ✅ the implementation cannot be initialized directly
- ✅ initialize can only run once
- ✅ initialize sets one 65-day phase selling 50,000,000 LRY at 0.075 USD

- ✅ initialize reverts if the KYC signer address is zero
- ✅ initialize transfers ownership to the owner
- ✅ buyTokens reverts without a valid KYC signature
- ✅ buyTokens reverts once the sale period has ended
- ✅ buyTokens reverts if the stablecoin balance is too low
- ✅ buyTokens reverts if the stablecoin allowance is too low
- ✅ buyTokens reverts on a zero payment
- ✅ buyTokens reverts if the payment buys zero LRY
- ✅ buyTokens reverts beyond the 50,000,000 LRY allocation
- ✅ buyTokens forwards native coin to the receiver address
- ✅ buyTokens moves USDT or USDC straight from the buyer to the receiver
- ✅ buyTokens sends the purchased LRY to the buyer
- ✅ buyTokens updates the phase sold amount, total sold and USD raised
- ✅ buyTokens applies the user's custom price when one is set
- ✅ buyTokens emits BuyTokenDetail with the amounts, type and phase
- ✅ buyTokens is protected against reentrancy
- ✅ calculateTokens matches what buyTokens credits in the same block
- ✅ getCurrentPhase returns the active phase
- ✅ receive forwards native coin to the receiver address
- ✅ sendLeftoverTokens is owner only
- ✅ sendLeftoverTokens reverts before the sale ends
- ✅ sendLeftoverTokens reverts on a second call
- ✅ sendLeftoverTokens reverts if every token was sold
- ✅ sendLeftoverTokens sends the unsold allocation to the receiver
- ✅ updateReceiverAddress is owner only and rejects the zero address
- ✅ setKycSigner is owner only and rejects the zero address
- ✅ setUserPrice is owner only and rejects an empty userId
- ✅ setUserPrice stores or clears the price and emits UserPriceUpdated

LendaryVesting

- ✔ the implementation cannot be initialized directly
- ✔ initialize can only run once
- ✔ initialize sets up the six vesting categories
- ✔ initialize sets the 200,000,000 LRY private-sale allocation
- ✔ initialize registers the five allocations totalling 715,000,000 LRY
- ✔ initialize reverts if the KYC signer address is zero
- ✔ registerPrivateSaleUser is owner only
- ✔ registerPrivateSaleUser reverts after the private sale ends
- ✔ registerPrivateSaleUser reverts on a zero amount
- ✔ registerPrivateSaleUser reverts on the zero address
- ✔ registerPrivateSaleUser reverts above the private-sale allocation
- ✔ registerPrivateSaleUser sends 10% now and vests 90% in category 1
- ✔ registerPrivateSaleUser emits RegisterPrivateSaleUser
- ✔ claimTokens reverts for an unregistered user
- ✔ claimTokens reverts when nothing is due yet
- ✔ claimTokens reverts once everything is claimed
- ✔ claimTokens pays only the slices that are due
- ✔ claimTokens pays the full remainder on the final slice
- ✔ claimTokens always sends the tokens to the registered user
- ✔ claimTokens emits ClaimedToken with the amount and slice count
- ✔ batchClaimFor reverts above 25 entries
- ✔ batchClaimFor reverts if the array lengths differ
- ✔ batchClaimFor claims every entry in one call
- ✔ tokensToBeClaimed returns zero before the category starts
- ✔ tokensToBeClaimed caps the slice count at the schedule length
- ✔ batchTokensToBeClaimed reverts if the array lengths differ
- ✔ sendLeftoverTokens is owner only
- ✔ sendLeftoverTokens reverts before the private sale ends

- ✔ sendLeftoverTokens reverts on a second call
- ✔ sendLeftoverTokens reverts if the allocation is fully used
- ✔ sendLeftoverTokens sends the unallocated private-sale LRY to the receiver
- ✔ updateReceiverAddress is owner only and rejects the zero address
- ✔ setKycSigner is owner only and rejects the zero address
- ✔ receive forwards native coin to the receiver address

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
LRY token (LRYToken)	Token	The asset that is sold, vested and staked. Staking pulls it on stake() and depositRewards() and pays it out on every exit and claim; ICO and Vesting transfer it from pre-funded balances. Its totalSupply() is the reference for every Staking anti-whale limit: position cap, velocity cap, whale multiplier and large-exit threshold.	Standard OpenZeppelin ERC20 with a fixed 1,000,000,000 supply minted once at deployment; no fees, hooks or rebasing.	Every anti-whale limit is a percentage of totalSupply(), so any change in supply shifts those limits for all stakers at once. Fee, hook or rebasing behaviour would break the 1:1 principal accounting in Staking.
stLRY receipt token	Token	Minted 1:1 on stake() and burned on every penalty and exit. Staking calls mint() and burn() on it, and stLRY only accepts those calls from its configured stakingContract.	stakingContract points at the deployed Staking proxy and is locked with lockStakingContract() before users stake.	While the pointer is unlocked, the stLRY owner decides who can mint and burn receipts. A wrong pointer makes every Staking mint and burn revert, blocking stakes and exits.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
USDT / USDC	Token	LendaryICO pulls the payment with safeTransferFrom from the buyer directly to receiverAddress, valuing one token at 1 USD scaled by its decimals().	Standard ERC20s that hold their 1 USD peg and transfer exactly the amount requested.	A depeg, a fee-on-transfer variant or a different decimals() would make the value received differ from the value used to price the purchase.
ETH/USD price feed (Chainlink)	Oracle	initialize() configures a Chainlink ETH/USD feed address (ETHOracle) as the price reference for native-coin purchases.	Native-coin purchases are priced at a current, correct ETH/USD rate for the deployment chain.	If native-coin pricing does not track the market, native-coin buyers are over- or under-charged relative to stablecoin buyers. A stale feed or a feed from another chain has the same effect.
KYC signer	Privileged role	Signs (userId, wallet, nonce, chainId, contract) for every ICO purchase and every stake. The contracts recover the signer with ECDSA and consume the wallet's nonce.	The key is kept secure and only signs after real KYC, for the correct userId and wallet.	A compromised key lets unverified wallets buy and stake and link wallets to arbitrary userIds. If the signer is unavailable, no new purchases or stakes can be made.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Backend userId registry	Privileged role	Staking groups positions, anti-whale limits and the governance bonus by a backend-issued userId string. The first signed stake links a wallet to that userId permanently.	The backend issues exactly one userId per real person and only signs links for wallets that person controls.	Wallets linked to the same userId share its positions, limits and bonus, so a wrong link gives another wallet control over those positions. Per-userId limits hold only if each person has a single userId.
Meta-transaction relayer	Library	executeMetaTransaction() verifies a signature over (nonce, chainId, contract, functionSignature), then self-calls with the user's address appended; _msgSender() reads that suffix in Staking, ICO and Vesting.	Relayers only submit payloads the user signed, and users only sign calls they intend.	A relayer can delay, reorder or withhold signed calls. Anyone holding a signed payload can submit it, so the signature is the only authorization.
Contract owner (Ownable)	Privileged role	Owns Staking, ICO, Vesting and LRYToken. Pauses Staking; sets the KYC signer, minimum stake, penalty and insurance receivers, governance bonus, auto-compound flag, custom user prices and sale receivers; rescues tokens and sweeps leftovers.	A multisig or timelock operated honestly.	A compromised or careless owner can pause exits, redirect penalties and proceeds, change reward parameters for all positions and set arbitrary custom prices. Losing the owner key removes all administration.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Proxy admin (TransparentUpgradeableProxy)	Interface	Staking, ICO and Vesting are deployed behind transparent proxies whose admin can replace the implementation at any time.	Every upgrade is audited and keeps the storage layout, and each proxy is initialized in the same transaction that deploys it.	The admin controls the logic over every balance the proxies hold. A faulty upgrade can corrupt storage, and an uninitialized proxy can be claimed by its first caller.
Sale receiver and vesting beneficiaries	Interface	ICO and Vesting send proceeds and leftovers to a receiverAddress set at initialize(). Vesting registers five allocation beneficiaries at fixed addresses during initialize().	The configured addresses are controlled by the project on the deployment chain.	Funds sent to these addresses cannot be recovered on-chain, so a wrong or lost address loses them.
Staking reward funding	Interface	Staking pays every reward from its own LRY balance. The owner tops it up with depositRewards().	The owner keeps enough LRY in Staking to cover all principal and all rewards.	Rewards and principal share one balance, so if funding falls short, payouts depend on the order in which users exit.
Sale and vesting funding	Token	LendaryICO must hold 50,000,000 LRY and LendaryVesting 915,000,000 LRY before purchases, registrations and claims. Both are funded by manual transfers after deployment.	Both contracts are funded with exactly their documented allocations before use.	An under-funded contract cannot complete the last purchases or claims. Tokens sent beyond the allocation are outside the contracts' accounting.

2. Entry & Exit Points Analysis (Function-Level)

This is the **core of the threat model**. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets **one table**.

Contract Name	Function	Category
LendaryStaking	`initialize()`	**Main invariant(s)** Runs once; all addresses and the minimum stake are non-zero. **Access level** Deployment (Public initializer. Must be called atomically by the proxy deployment.) **What this function can do** Set the LRY and stLRY tokens, minimum stake, KYC signer, owner, penalty receiver and insurance reserve, and write the three tier configurations (lock-up, cooldown, base APY, max APY, early-exit penalty). **What this function cannot/should not do** It should not be callable twice or by anyone other than the deployer.

Contract Name	Function	Category
LendaryStaking	`stake()`	**Main invariant(s)** Staked principal equals minted stLRY. A userId's aggregate stake stays within 2% of supply, and new stake within one window stays within 0.5% of supply. **Access level** Public (Public, while not paused; requires a valid KYC signature.) **What this function can do** Verify and consume the caller's KYC signature, link the wallet to the userId on first use, enforce the 2% position cap and the 0.5% per-window velocity cap, pull LRY, mint stLRY 1:1, open a new position and refresh the whale multiplier of every position of the userId. **What this function cannot/should not do** It should not accept Base tier with auto-compound, amounts below the minimum stake, an empty userId, or a wallet already linked to another userId.
LendaryStaking	`unstake()`	**Main invariant(s)** Principal moves into cooldown unchanged; no LRY leaves the contract. **Access level** Public (Any wallet linked to the position's userId, while not paused.) **What this function can do** After lock end, book pending rewards (running monthly auto-compound when active) and move the whole principal into cooldown, using the tier cooldown or the large-exit cooldown above 1% of supply. **What this function cannot/should not do** It should not run before lock end or on a position that is already closed or in cooldown.

Contract Name	Function	Category
LendaryStaking	`unstakeEarly()`	**Main invariant(s)** Penalty plus forfeited rewards go to penaltyReceiver; principal minus penalty goes into cooldown. **Access level** Public (Any wallet linked to the position's userId, while not paused.) **What this function can do** Before lock end, book rewards, take the tier's early-exit penalty (3%, 4% or 5% of principal), forfeit booked rewards to the penalty receiver, burn the penalty's stLRY and move the remainder into cooldown. **What this function cannot/should not do** It should not run after lock end, and it should not let the staker keep rewards earned during the lock.
LendaryStaking	`partialUnstake()`	**Main invariant(s)** Amount is strictly below principal; principal stays non-zero. **Access level** Public (Any wallet linked to the position's userId, while not paused.) **What this function can do** For Premium positions at least 90 days old, take a 2.5% penalty on the slice to the penalty receiver, send the slice's share of booked rewards to the insurance reserve, and move the slice minus penalty into cooldown. **What this function cannot/should not do** It should not apply to non-Premium positions, positions younger than 90 days, or the full principal.

Contract Name	Function	Category
LendaryStaking	`withdraw()`	**Main invariant(s)** Paid principal equals burned stLRY; _totalObligated decreases by the amount paid. **Access level** Public (Any wallet linked to the position's userId, while not paused.) **What this function can do** After the cooldown ends, book rewards, burn the cooldown principal's stLRY and pay the cooldown principal, plus all booked rewards if the position is fully closed. **What this function cannot/should not do** It should not pay before the cooldown ends or pay the same principal twice.
LendaryStaking	`accelerateCooldown()`	**Main invariant(s)** Fee bands are fixed; net payout plus fee equals the cooldown principal. **Access level** Public (Any wallet linked to the position's userId, while not paused.) **What this function can do** End a standard (non-large) cooldown early: charge a 0.5% to 2% fee on the principal depending on time remaining, pay principal net of fee plus booked rewards, and send the fee to the insurance reserve. **What this function cannot/should not do** It should not be available for large exits or after the cooldown has already ended.

Contract Name	Function	Category
LendaryStaking	`claimMonthlyRewards()`	**Main invariant(s)** Booked rewards drop to zero; the claim timestamp becomes the new monthly checkpoint. **Access level** Public (Any wallet linked to the position's userId, while not paused.) **What this function can do** Book rewards and pay all booked rewards to the caller, at most once per month, or only after lock end for positions with auto-compound active. **What this function cannot/should not do** It should not pay twice for the same period.
LendaryStaking	`depositRewards()`	**Main invariant(s)** Only the contract's LRY balance changes; _totalObligated is untouched. **Access level** Owner **What this function can do** Pull LRY from the owner into Staking to fund future rewards. **What this function cannot/should not do** It should not change any position or obligation.
LendaryStaking	`rescueERC20()`	**Main invariant(s)** LRY balance stays at or above _totalObligated. Other tokens are unrestricted. **Access level** Owner **What this function can do** Transfer any token held by Staking to an owner-chosen address. For LRY, the balance must stay at or above _totalObligated after the transfer. **What this function cannot/should not do** It should never take LRY that backs staked principal or booked rewards.

Contract Name	Function	Category
LendaryStaking	`pause() / unpause()`	**Main invariant(s)** No balances change. **Access level** Owner **What this function can do** Stop or resume stake, unstake, unstakeEarly, partialUnstake, withdraw, accelerateCooldown and claimMonthlyRewards. **What this function cannot/should not do** It should not be used to hold user funds indefinitely.
LendaryStaking	`setKycSigner() / setMinStake()`	**Main invariant(s)** Both values stay non-zero; existing positions are unaffected. **Access level** Owner **What this function can do** Replace the KYC signer, or change the minimum stake for new positions. **What this function cannot/should not do** It should not set a zero signer or a zero minimum.
LendaryStaking	`updatePenaltyReceiver() / updateInsuranceReserve()`	**Main invariant(s)** Both sinks stay non-zero. **Access level** Owner **What this function can do** Redirect where penalties, forfeited rewards and acceleration fees are sent. **What this function cannot/should not do** It should not set either sink to address(0).

Contract Name	Function	Category
LendaryStaking	`submitGovernanceEligible()`	**Main invariant(s)** Each listed userId's bonus end is set to now plus 90 days. **Access level** Owner **What this function can do** Grant listed userIds a +1% APY bonus for 90 days. **What this function cannot/should not do** It should not change rewards for time before the grant.
LendaryStaking	`setMonthlyAutoCompoundEnabled()`	**Main invariant(s)** Only the global flag changes. **Access level** Owner **What this function can do** Turn monthly auto-compounding on or off for every position that opted in. **What this function cannot/should not do** It should not re-price months that were already earned.
LendaryStaking	`pendingRewards()` / `pendingRewardsForStakeIds()`	**Main invariant(s)** View only. **Access level** Public **What this function can do** Simulate accrual, including monthly compounding, and return the rewards a userId or a list of positions would receive. **What this function cannot/should not do** They should not change state, and they should match what a claim would pay.

Contract Name	Function	Category
LendaryStaking	`getWalletsByUserId()` / `getStakeIdsByWallet()` / `getStakeIdsByUserId()` / `aggregatedStakedOf()` / `totalStakedByWallet()` / `effectiveAPYBps()` / `currentWhaleMultiplierBps()` / `partialUnstakeEarliest()`	**Main invariant(s)** View only. **Access level** Public **What this function can do** Return position, wallet-link, aggregate-stake, APY and multiplier data. **What this function cannot/should not do** They should not change state.
LendaryICO	`initialize()`	**Main invariant(s)** Runs once; totalPhases is 1. **Access level** Deployment (Public initializer. Must be called atomically by the proxy deployment.) **What this function can do** Set LRY, USDT and USDC, the price feed and receiver, a single 65-day phase selling 50,000,000 LRY at 0.075 USD, the KYC signer and the owner. **What this function cannot/should not do** It should not run twice, accept zero addresses, or be callable by an arbitrary address.

Contract Name	Function	Category
LendaryICO	`buyTokens()`	**Main invariant(s)** totalTokenSold equals the phase's tokenSold and never exceeds the phase limit; sold counters only increase. **Access level** Public (Public, nonReentrant; requires a valid KYC signature.) **What this function can do** Verify and consume the KYC signature, value the payment in USD (native coin or a stablecoin) at the phase price or the user's custom price, update the sold counters, forward the payment to receiverAddress and send the purchased LRY to the buyer. **What this function cannot/should not do** It should not sell beyond 50,000,000 LRY, after the phase ends or after the sweep, or accept a payment that does not match the declared payment type.
LendaryICO	`receive()`	**Main invariant(s)** The ICO's own native-coin balance is unchanged. **Access level** Public **What this function can do** Forward any plain native-coin transfer to receiverAddress. **What this function cannot/should not do** It should not retain native coin in the ICO.

Contract Name	Function	Category
LendaryICO	`sendLeftoverTokens()`	**Main invariant(s)** Runs at most once; sends the unsold allocation. **Access level** Owner **What this function can do** After the phase expires, send tokensToBeSold minus totalTokenSold LRY to receiverAddress, once. **What this function cannot/should not do** It should not run before the sale ends or run twice.
LendaryICO	`updateReceiverAddress()`	**Main invariant(s)** Receiver stays non-zero. **Access level** Owner **What this function can do** Change the address that receives sale proceeds and the leftover sweep. **What this function cannot/should not do** It should not accept an invalid receiver such as address(0) or the ICO itself.
LendaryICO	`setKycSigner()`	**Main invariant(s)** Signer stays non-zero. **Access level** Owner **What this function can do** Replace the KYC signer used by buyTokens(). **What this function cannot/should not do** It should not set a zero signer.

Contract Name	Function	Category
LendaryICO	`setUserPrice()`	**Main invariant(s)** Only the userId's override changes. **Access level** Owner **What this function can do** Set a custom per-userId price that overrides the phase price, or clear it with 0. **What this function cannot/should not do** It should not set a price outside the range the sale intends to offer.
LendaryICO	`getCurrentPhase()` / `calculateTokens()`	**Main invariant(s)** View only; the quote matches what buyTokens() credits in the same block. **Access level** Public **What this function can do** Return the active phase and quote LRY for a payment amount and type. **What this function cannot/should not do** They should not change state.
LendaryVesting	`initialize()`	**Main invariant(s)** Runs once; vesting schedules are fixed after it. **Access level** Deployment (Public initializer. Must be called atomically by the proxy deployment.) **What this function can do** Set LRY, the receiver and five beneficiaries, six vesting categories, the 200,000,000 LRY private-sale allocation and its end time, register 715,000,000 LRY to the beneficiaries, and set the KYC signer and owner. **What this function cannot/should not do** It should not run twice, accept zero addresses, or be callable by an arbitrary address.

Contract Name	Function	Category
LendaryVesting	`registerPrivateSaleUser()`	**Main invariant(s)** privateSaleAllocated never exceeds privateSaleAllocation. **Access level** Owner (Owner only, nonReentrant.) **What this function can do** Before the private sale ends, credit a buyer: 10% of the amount is transferred immediately and 90% vests in category 1 over 12 monthly slices. **What this function cannot/should not do** It should not exceed the 200,000,000 LRY private-sale allocation or register after the sale ends.
LendaryVesting	`claimTokens()`	**Main invariant(s)** claimedAmount only increases and never exceeds totalAmount. **Access level** Public (Public: anyone can trigger a claim, but tokens always go to the registered user.) **What this function can do** Pay a registered user the slices that have become due in a category. **What this function cannot/should not do** It should not pay more than the user's total or pay the same slice twice.
LendaryVesting	`batchClaimFor()`	**Main invariant(s)** Equivalent to calling claimTokens() for each pair. **Access level** Public **What this function can do** Claim for several (user, category) pairs in one call. **What this function cannot/should not do** It should not pay any user more than is due.

Contract Name	Function	Category
LendaryVesting	`sendLeftoverTokens()`	**Main invariant(s)** Runs at most once; sends privateSaleAllocation minus privateSaleAllocated. **Access level** Owner **What this function can do** After the private sale ends, send the unallocated private-sale LRY to receiverAddress, once. **What this function cannot/should not do** It should not run before the sale ends or run twice.
LendaryVesting	`updateReceiverAddress()` / `setKycSigner()`	**Main invariant(s)** Both stay non-zero. **Access level** Owner **What this function can do** Change the leftover receiver, or the stored KYC signer. **What this function cannot/should not do** It should not accept an invalid receiver such as address(0) or Vesting itself.
LendaryVesting	`receive()`	**Main invariant(s)** Vesting's own native-coin balance is unchanged. **Access level** Public **What this function can do** Forward any plain native-coin transfer to receiverAddress. **What this function cannot/should not do** It should not retain native coin in Vesting.

Contract Name	Function	Category
LendaryVesting	`tokensToBeClaimed()` / `batchTokensToBeClaimed()`	**Main invariant(s)** View only. **Access level** Public **What this function can do** Return the amount and slice count currently claimable. **What this function cannot/should not do** They should not change state.
Staking / ICO / Vesting	`executeMetaTransaction()`	**Main invariant(s)** The user's nonce increases by at least one per successful call. **Access level** Public (Public relayer entry point.) **What this function can do** Verify a user's signature over the call, increment the user's nonce, and execute the call as the user by appending their address to the calldata. **What this function cannot/should not do** It should not accept the same signature twice or execute a call the user did not sign.
Staking / ICO / Vesting	`getNonce()` / `verify()` / `getChainID()` / `kycSignDigest()`	**Main invariant(s)** View only. **Access level** Public **What this function can do** Return the meta-transaction nonce, check a meta-transaction signature, return the chain id, and return the exact digest the KYC signer must sign. **What this function cannot/should not do** They should not change state.

Contract Name	Function	Category
Staking / ICO / Vesting	`transferOwnership()` / `renounceOwnership()`	**Main invariant(s)** Exactly one owner, or none after renounce. **Access level** Owner **What this function can do** Hand ownership to a new address, or remove the owner entirely. **What this function cannot/should not do** It should not hand ownership to an address that cannot use it.
stLRY	`setStakingContract()` / `lockStakingContract()`	**Main invariant(s)** After locking, stakingContract never changes. **Access level** Owner **What this function can do** Point stLRY at the Staking contract, then lock the pointer permanently. **What this function cannot/should not do** It should not be possible to repoint the minter once users hold stLRY.
stLRY	`mint()` / `burn()`	**Main invariant(s)** totalSupply() equals the sum of all balances. **Access level** Contract only **What this function can do** Mint receipts on stake and burn them on penalties and exits. **What this function cannot/should not do** They should not be callable by anyone except Staking.

Contract Name	Function	Category
stLRY	`transfer()` / `transferFrom()` / `approve()` / `increaseAllowance()` / `decreaseAllowance()`	**Main invariant(s)** Balances only change through mint and burn. **Access level** Public (Public; always reverts.) **What this function can do** Always revert: stLRY is non-transferable. **What this function cannot/should not do** They should never move receipts between accounts.
stLRY	`transferOwnership()` / `acceptOwnership()` / `renounceOwnership()`	**Main invariant(s)** Ownership changes only on acceptOwnership() or renounceOwnership(). **Access level** Owner (Owner, or the pending owner for acceptOwnership().) **What this function can do** Two-step ownership handover, or renounce. **What this function cannot/should not do** It should not hand over ownership without acceptance.
LRYToken	`transfer()` / `transferFrom()` / `approve()`	**Main invariant(s)** totalSupply() equals the sum of all balances. **Access level** Public **What this function can do** Standard ERC20 transfers and allowances. **What this function cannot/should not do** They should not change totalSupply().

Contract Name	Function	Category
LRYToken	`burn() / burnFrom()`	**Main invariant(s)** totalSupply() only decreases and never exceeds 1,000,000,000 LRY. **Access level** Public **What this function can do** Burn the caller's LRY, or LRY the caller is allowed to spend. **What this function cannot/should not do** It should not burn tokens the caller does not own or is not allowed to spend.
LRYToken	`transferOwnership() / renounceOwnership()`	**Main invariant(s)** The owner cannot mint or move tokens; ownership is inert. **Access level** Owner **What this function can do** Hand over or renounce the token's ownership. **What this function cannot/should not do** The owner should hold no special token power.

3. Asset Flow Mapping (Critical Contracts)

This section identifies **where money actually lives** and **how it moves**.

3.1 Asset-Holding Contracts

List only contracts that **custody value**.

Contract	Asset Type	Custodied Assets
LendaryStaking	LRY token	All staked principal, active and in cooldown, every booked reward, and the reward float the owner deposits, held in one LRY balance.
LendaryICO	LRY token; native coin and stablecoins in transit	The 50,000,000 LRY sale allocation until it is bought or swept. Native-coin and stablecoin payments pass straight through to receiverAddress.
LendaryVesting	LRY token	The 915,000,000 LRY vesting pool: the 200,000,000 LRY private-sale allocation plus 715,000,000 LRY registered to the five beneficiaries, released in monthly slices.

3.2 Asset Entry & Exit Functions

This table maps **money movement**, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Lendary Staking	`stake()`	LRY from the staker.	stLRY minted 1:1 to the staker.	Public	Anti-whale limits are measured against LRY totalSupply() and grouped per userId, so they depend on correct userId assignment. Caller note: Any wallet with a valid KYC signature.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Lendary Staking	`depositRewards()`	LRY from the owner.	–	Owner	Rewards share one balance with principal, so solvency depends on adequate funding.
Lendary Staking	`unstakeEarly()`	–	Penalty plus forfeited rewards to penaltyReceiver; the remainder moves into cooldown.	Public	Value leaves the position immediately; every wallet linked to the userId can trigger it. Caller note: Any wallet linked to the position's userId.
Lendary Staking	`partialUnstake()`	–	2.5% penalty to penaltyReceiver; the slice's share of booked rewards to insuranceReserve; slice minus penalty into cooldown.	Public	Cheaper than unstakeEarly(), so the slice limits decide how much principal can leave at the reduced rate. Caller note: Any wallet linked to the position's userId; Premium only, after 90 days.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Lendary Staking	`withdraw()`	–	Cooldown principal, plus booked rewards when the position is closed, to the caller; stLRY burned.	Public	Paid from the shared balance; requires Staking to hold principal plus booked rewards. Caller note: Any wallet linked to the position's userId, after the cooldown.
Lendary Staking	`accelerateCooldown()`	–	Principal net of a 0.5% to 2% fee, plus booked rewards, to the caller; the fee to insuranceReserve.	Public	The fee is a percentage of principal, so rounding reduces it on very small amounts. Caller note: Any wallet linked to the position's userId; standard (non-large) exits only.
Lendary Staking	`claimMonthlyRewards()`	–	Booked rewards to the caller.	Public	Paid from the shared balance; claim frequency is enforced by timestamp checkpoints. Caller note: Any wallet linked to the position's userId; monthly, or after lock end for auto-compound positions.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Lendary Staking	`rescueERC20()`	–	Any token to an owner-chosen address; LRY only above _totalObligated.	Owner	LRY rescue is bounded by _totalObligated, so that counter must track principal and rewards exactly. Other tokens are unrestricted.
LendaryICO	`buyTokens()`	Native coin (forwarded to receiverAddress) or USDT/USDC (pulled straight from the buyer to receiverAddress).	LRY to the buyer, immediately.	Public	Proceeds go straight to receiverAddress. The LRY amount depends on the payment type, the price source and any per-user custom price. Caller note: Any buyer with a valid KYC signature.
LendaryICO	`receive()`	Native coin.	Forwarded to receiverAddress.	Public	Native coin is forwarded without being linked to a purchase.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LendaryICO	`sendLeftoverTokens()`	–	tokensToBeSold minus totalTokenSold LRY to receiverAddress.	Owner	The amount comes from sale counters, not from the actual balance. Caller note: Owner only; once, after the sale ends.
Lendary Vesting	`registerPrivateSaleUser()`	–	10% of the amount to the buyer immediately; the remaining 90% is scheduled in category 1.	Owner	Bounded by the 200,000,000 LRY private-sale allocation. Caller note: Owner only, before the private sale ends.
Lendary Vesting	`claimTokens()` / `batchClaimFor()`	–	Due slices to the registered user.	Public	Anyone can trigger a claim; payouts always go to the registered user. Caller note: Anyone, on the user's behalf.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Lendary Vesting	`sendLeftoverTokens()`	—	privateSaleAllocation minus privateSaleAllocated LRY to receiverAddress.	Owner	The amount comes from allocation counters, not from the actual balance; the receiver must be a valid external address. Caller note: Owner only; once, after the private sale ends.
Lendary Vesting	`receive()`	Native coin.	Forwarded to receiverAddress.	Public	Native coin is forwarded and not tracked.
Staking / ICO / Vesting	`executeMetaTransaction()`	Native coin sent as msg.value.	None: the relayed self-call forwards no value.	Public	Relayed calls run with the signer's identity, so a valid signature is the only authorization.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Lendary. We performed our audit according to the procedure described above. 2 High, 6 Medium, 2 Low and 4 Informational severity issues were found. Of these, the Lendary Team fixed 13 and acknowledged 1.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With 8+ years of expertise, we've secured 1500+ projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey



AUDIT REPORT

October 2026

For

Lendary

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com